

# 스마트 AI 카트

## 프론트엔드 개발 계획서

팀 카트라이더 · 프론트엔드 파트

담당자: 최수환 · 김도영

본 문서는 스마트 AI 카트 프로젝트의 모바일 앱 프론트엔드 개발 계획을 정리한 것이다. 사용자가 카트와 QR로 연동한 뒤 실시간 장바구니를 확인하고 결제까지 완료하는 전체 흐름을 구현한다. 기술 선택은 화면 5개, REST 엔드포인트 약 5개, 실시간 채널 1개라는 실제 규모에 맞게 과잉 의존성을 배제하고 최소한의 스택으로 구성한다.

### 1. 사용 기술 목록과 역할

| 기술                      | 역할                                                       |
|-------------------------|----------------------------------------------------------|
| React Native + Expo SDK | iOS/Android 크로스플랫폼 앱 기반. Expo가 빌드·배포·네이티브 API를 담당한다.     |
| Expo Router v3          | 파일 기반 라우팅. app/ 폴더 구조로 화면 전환을 선언적으로 관리한다.                |
| TypeScript              | 전체 코드에 타입 적용. 백엔드 API 응답 형식과의 불일치를 컴파일 단계에서 감지한다.        |
| Context + useReducer    | 전역 상태 관리. 장바구니·카트 연결 상태·인증 정보를 앱 전역에서 공유한다.              |
| Custom fetch wrapper    | JWT 자동 첨부·에러 공통 처리를 직접 구현한 HTTP 클라이언트 (~20줄).            |
| WebSocket (내장 API)      | Spring Boot와 실시간 채널 유지. 카트 인식 결과를 수신해 장바구니를 즉시 갱신한다.     |
| expo-camera             | AI 인식 실패 시 사용자가 직접 바코드를 스캔하는 수동 보정 수단. 카트 QR 연동에도 활용 가능. |
| expo-secure-store       | JWT 토큰 안전 저장. 앱 재시작 후에도 로그인 상태를 유지한다.                    |

### 경량화 원칙 — 제거한 라이브러리

아래 세 가지는 이 프로젝트 규모에서 필요하지 않다고 판단하여 배제하였다.

| 제거 대상          | 제거 이유                                                               | 대체 방법         |
|----------------|---------------------------------------------------------------------|---------------|
| Axios          | fetch 내장 API로 대체 가능. Custom fetch wrapper 20줄로 JWT 인터셉터까지 처리한다.     | 번들 크기 50KB 절감 |
| TanStack Query | 실시간 데이터는 WebSocket이 담당. REST 호출은 단순 mutation 5개 수준으로 캐싱 레이어가 불필요하다. | 초기 복잡도 감소     |
| Zustand        | 전역 상태가 장바구니·인증 두 가지뿐. React 내장 Context + useReducer로 충분하다.          | 외부 의존성 제거     |

## 2. 화면 구성 및 기능 명세

| 화면      | 주요 기능                                                               | 통신 방식            |
|---------|---------------------------------------------------------------------|------------------|
| 카트 연동   | expo-camera로 QR 스캔 → 카트 ID 추출 → Spring Boot 연동 요청 → WebSocket 연결 수립 | REST (fetch)     |
| 장바구니    | WebSocket 수신으로 상품 목록·합계 자동 갱신. 수동 수정·수량 변경은 별도 REST 호출              | WebSocket + REST |
| 결제 확인   | 주문 내역 요약 표시. 결제 수단 확인 후 1탭으로 최종 확인 요청 전송                            | REST (fetch)     |
| 결제 완료   | 결제 완료 화면, QR 영수증 발급, 음성 안내 제공. 카트 연결 세션 종료                          | —                |
| 매장 길 안내 | 상품 위치를 지도 형태로 안내. 카트 근처 할인 이벤트 알림 병행 표시                             | REST (fetch)     |

### UX 원칙 (노약자 친화)

- 글자 크기 18pt 이상, 주요 버튼·금액 표시 22~30pt 적용
- WCAG 2.1 AA 기준 고대비 색상 (색상 대비율 4.5:1 이상)
- 모든 인터랙션 요소 최소 높이 56px — 손 떨림 오조작 방지
- 결제 전체 동선 2탭 이내 완결 (결제 확인 1탭 → 결제 완료 1탭)
- 결제 완료 화면 음성 안내 제공 — 시각 의존도 완화

## 3. 백엔드 연동 구조

앱이 직접 통신하는 서버는 Spring Boot 단일 창구뿐이다. FastAPI·Kafka·Redis·라즈베리파이 는 내부 파이프라인에서만 동작하며 앱이 이를 인식하지 않는다.

| 통신 방향           | 방식               | 용도                                                             |
|-----------------|------------------|----------------------------------------------------------------|
| Spring Boot → 앱 | WebSocket        | 카트 카메라 인식 결과를 실시간으로 앱에 푸시. 장바구니 화면이 즉시 갱신된다.                   |
| 앱 → Spring Boot | REST API (HTTPS) | 카트 연동 요청, 결제 요청, 수동 수정, 수량 변경 등 사용자의 능동적 액션 전달.                |
| 앱이 인식하지 않는 내부   | 내부 전용            | FastAPI (AI 인식), Kafka (이벤트 큐), Redis (장바구니 캐시), MySQL (영구 저장) |

커스텀 fetch 래퍼는 expo-secure-store에서 JWT를 읽어 모든 REST 요청 헤더에 자동 첨부한다. WebSocket 연결 시 인증 방식(쿼리 파라미터 vs STOMP 헤더)은 본선 진출 확정 후 백엔드 파트와 협의하여 확정한다.

## 4. 개발 일정

| 작업 내용                           | 5월 | 6월 | 7월 | 8월 | 9월 |
|---------------------------------|----|----|----|----|----|
| 기술 스택 확정 및 프로젝트 초기 세팅           | ●  |    |    |    |    |
| 백엔드 API 명세 협의 (WebSocket·인증 방식) | ●  | ●  |    |    |    |
| 카트 연동 화면 구현 (QR 스캔·연결)          |    | ●  | ●  |    |    |
| 장바구니 실시간 화면 구현 (WebSocket 연동)   |    |    | ●  | ●  |    |
| 결제 확인·완료 화면 구현                  |    |    | ●  | ●  |    |
| 매장 길 안내 화면 구현                   |    |    | ●  | ●  |    |
| 노약자 친화 UX 검증 및 접근성 개선           |    |    |    | ●  | ●  |
| 백엔드 통합 테스트 및 버그 수정              |    |    |    | ●  | ●  |
| 시연 영상·발표자료 대응                   |    |    |    | ●  | ●  |

※ ● 주력 진행 / ● 병행 진행

※ 본선 진출 확정 후 6월 중 백엔드 API 명세를 공유받아 WebSocket 연결 방식 및 REST 엔드포인트를 협의하고 본격 구현에 착수한다.